Guía práctica de Conventional Commits

Buenas prácticas para equipos de desarrollo de software

1. ¿Qué son los Conventional Commits?

Conventional Commits es una convención para escribir mensajes de commit de manera estructurada, clara y consistente. Permite que el historial de Git sea fácil de leer, mantener y revisar por cualquier integrante de un equipo de desarrollo.

La estructura básica es:

```
tipo: descripción breve del cambio
```

Ejemplo:

```
feat: agregar autenticacion de usuarios
```

2. ¿Por qué utilizar una convención?

- Hace que el historial del repositorio sea más comprensible.
- Permite identificar rápidamente la naturaleza de cada cambio.
- Facilita las revisiones de código y el trabajo colaborativo.
- Ayuda a encontrar cambios relacionados con funcionalidades o correcciones.
- Reduce mensajes ambiguos o poco útiles como “cambios” o “avance”.
- Puede facilitar procesos automatizados de versionado, changelogs y releases.

3. Estructura de un commit convencional

La forma más sencilla de utilizar Conventional Commits consiste en indicar primero el tipo de cambio y después una descripción breve y específica.

```
tipo: descripcion del cambio
```

Ejemplos:

```
feat: agregar modulo de usuarios

fix: corregir validacion del formulario

docs: actualizar instrucciones de instalacion

refactor: simplificar servicio de autenticacion
```

4. Tipos de commit más utilizados

Tipo	Significado	Cuándo usarlo	Ejemplo
feat	Feature / funcionalidad	Cuando se agrega una capacidad nueva al sistema.	feat: agregar recuperacion de contraseña
fix	Corrección	Cuando se corrige un error o comportamiento incorrecto.	fix: corregir validacion del correo
chore	Mantenimiento	Para tareas de mantenimiento, configuración o infraestructura que no agregan una funcionalidad de negocio.	chore: actualizar dependencias
docs	Documentación	Para documentación, README, guías o documentación técnica.	docs: actualizar guia de instalacion
refactor	Refactorización	Para mejorar la estructura interna del código sin cambiar su comportamiento.	refactor: simplificar servicio de usuarios
test	Pruebas	Para agregar o modificar pruebas automatizadas.	test: agregar pruebas de autenticacion
style	Estilo	Para cambios de formato que no afectan la lógica.	style: aplicar formato al codigo
perf	Rendimiento	Para mejoras de rendimiento.	perf: optimizar consulta de usuarios
build	Build / compilación	Para cambios relacionados con compilación o herramientas de construcción.	build: actualizar configuracion de compilacion
ci	Integración continua	Para cambios en CI/CD y automatizaciones del repositorio.	ci: agregar validacion automatica

5. Los tipos principales explicados

feat

Se utiliza cuando el producto o sistema obtiene una nueva funcionalidad.

```
feat: agregar registro de usuarios

feat: implementar busqueda de productos

feat: agregar exportacion de reportes
```

fix

Se utiliza cuando se corrige algo que ya existía pero no funcionaba correctamente.

```
fix: corregir error al iniciar sesion

fix: corregir calculo del total

fix: solucionar error de conexion
```

chore

Se utiliza para tareas de mantenimiento, configuración o soporte del proyecto.

```
chore: actualizar dependencias

chore: configurar variables de entorno

chore: reorganizar archivos de configuracion
```

docs

Se utiliza para cambios en documentación.

```
docs: actualizar README

docs: agregar guia de instalacion

docs: documentar API
```

refactor

Se utiliza cuando se mejora la estructura del código sin cambiar su comportamiento esperado.

```
refactor: reorganizar servicio de usuarios

refactor: simplificar metodo de validacion
```

test

Se utiliza para agregar o modificar pruebas.

```
test: agregar pruebas de usuarios

test: cubrir escenario de autenticacion
```

6. ¿Cómo elegir el tipo correcto?

```
¿Estoy agregando una funcionalidad nueva? → feat

¿Estoy corrigiendo un error? → fix

¿Estoy reorganizando/mejorando código sin cambiar su comportamiento? → refactor

¿Estoy modificando documentación? → docs

¿Estoy agregando o modificando pruebas? → test

¿Estoy realizando configuración o mantenimiento? → chore

¿Estoy cambiando solamente el formato del código? → style

¿Estoy mejorando el rendimiento? → perf
```

```
¿Estoy modificando compilación o dependencias de build? → build

¿Estoy modificando CI/CD? → ci
```

7. Buenas prácticas para escribir mensajes

- Escribir el tipo en minúsculas.
- Mantener la descripción breve pero específica.
- Describir el cambio realizado, no solamente el archivo modificado.
- Preferir verbos en infinitivo: agregar, corregir, actualizar, implementar, configurar.
- Mantener una convención uniforme dentro del equipo.
- Crear commits que representen una unidad lógica de trabajo.
- Evitar mezclar cambios no relacionados en un mismo commit.

8. Ejemplos de buenos mensajes

```
feat: agregar autenticacion de usuarios

feat: implementar filtro por fecha

fix: corregir validacion de contraseña

fix: solucionar error al guardar registros

chore: actualizar dependencias del proyecto

docs: agregar instrucciones de despliegue

refactor: simplificar servicio de clientes

test: agregar pruebas para registro de usuarios

style: aplicar formato al codigo

perf: optimizar consulta de reportes
```

9. Ejemplos de mensajes que conviene evitar

```
cambios

avance

actualizacion

correcciones

trabajo de hoy

ya funciona

final

prueba
```

Estos mensajes no explican suficientemente qué ocurrió y pierden valor cuando alguien revisa el historial semanas o meses después.

10. Un commit debe representar una unidad lógica

No es recomendable crear un commit por cada línea modificada. Tampoco es recomendable crear un único commit gigantesco que mezcle funcionalidades, correcciones, documentación y configuración sin relación entre sí.

Ejemplo de una secuencia coherente:

```
feat: agregar modulo de usuarios

feat: implementar validacion de usuarios

test: agregar pruebas de usuarios

fix: corregir validacion de correo

docs: documentar modulo de usuarios
```

11. Verbos recomendados

Para mantener los mensajes consistentes, pueden utilizar verbos como:

- agregar
- implementar
- crear
- configurar
- actualizar
- corregir
- eliminar
- reorganizar
- optimizar
- documentar
- refactorizar
- validar

12. Ejemplos de comandos Git

Revisar los cambios:

```
git status
```

Preparar los archivos modificados:

```
git add .
```

Crear un commit:

```
git commit -m "tipo: descripcion del cambio"
```

Subir los commits al repositorio remoto:

```
git push
```

El nombre de la rama y el flujo de integración dependen de la estrategia de ramas utilizada por cada equipo.

13. Conventional Commits y trabajo colaborativo

La convención es especialmente útil en equipos porque todos escriben los commits utilizando un lenguaje común. Esto facilita revisar el historial, identificar cambios y comprender qué trabajo se realizó.

Un historial claro podría verse así:

```
chore: inicializar repositorio

feat: agregar modulo de autenticacion

feat: implementar registro de usuarios

test: agregar pruebas de autenticacion

fix: corregir validacion de contraseña

refactor: simplificar servicio de autenticacion

docs: actualizar documentacion de instalacion
```

14. Formato opcional para casos más complejos

Conventional Commits también permite especificar un ámbito (scope) para indicar qué parte del sistema fue afectada.

```
tipo(ambito): descripcion
```

Ejemplos:

```
feat(auth): agregar recuperacion de contraseña

fix(api): corregir respuesta de usuarios

docs(readme): actualizar instrucciones
```

El uso de scopes es opcional. Lo importante es que el equipo acuerde una convención y la mantenga de forma consistente.

15. Regla práctica de oro

Un buen commit debe permitir que otra persona lea el historial y entienda qué cambió sin tener que abrir todos los archivos del proyecto.

Formato recomendado:

```
tipo: descripcion breve y especifica del cambio
```

En resumen: usa el tipo correcto, describe el cambio con claridad, mantén cada commit enfocado y utiliza la misma convención en todo el equipo.